

Deep XSS Payload Notes & Methodology Guide

1. What is XSS?

XSS (Cross Site Scripting) is a web vulnerability where attackers inject JavaScript into webpages viewed by other users.

Goals:

- Session hijacking
- Cookie theft
- Phishing
- Keylogging
- Account takeover
- Browser exploitation

Core Formula:

CONTEXT → ESCAPE → EXECUTE

2. Types of XSS

Reflected XSS:

Payload comes from request and reflects immediately.

Stored XSS:

Payload is stored in database and executes later for users.

DOM XSS:

JavaScript on browser side causes execution.

Stored DOM XSS:

Stored payload later inserted into dangerous DOM sink.

3. Understanding Contexts

Payloads depend completely on context.

HTML Context:

```
<div>INPUT</div>
```

Attribute Context:

```
<input value="INPUT">
```

JavaScript Context:

```
var x = 'INPUT';
```

Textarea Context:

```
<textarea>INPUT</textarea>
```

Select Context:

```
<select><option>INPUT</option></select>
```

Every context requires different breakout methods.

4. Script Tag Payload

Payload:

```
<script>alert(1)</script>
```

Purpose:

Direct JavaScript execution.

How It Works:

Browser parses script tag and executes JavaScript inside it.

When It Works:

- HTML context
- No filtering
- Script tags allowed

When It Fails:

- Angle brackets encoded
- CSP blocks inline scripts
- HTML sanitized

5. IMG onerror Payload

Payload:

```
<img src=x onerror=alert(1)>
```

Breakdown:

<img

Creates image element.

src=x

Invalid image path.

onerror=

Triggered when image loading fails.

alert(1)

JavaScript executes.

Execution Flow:

Browser loads image → image fails → onerror triggers → JS executes.

Advantages:

- Short
- Reliable
- Works without script tag
- Bypasses many filters

Common Uses:

- HTML injection
- DOM XSS
- Stored XSS

6. SVG onload Payload

Payload:

```
<svg onload=alert(1)>
```

Why SVG?

SVG supports JavaScript event handlers.

How It Works:

SVG loads immediately → onload executes.

Advantages:

- Works where script tags blocked
- Often bypasses filters
- Common DOM XSS payload

Common Use Cases:

- innerHTML sinks
- HTML injection
- CSP bypass testing

7. Attribute Injection Payload

Payload:
" autofocus onfocus=alert(1) x="

Context:
<input value="INPUT">

Goal:
Break attribute and inject new event handler.

Breakdown:

"
Closes existing attribute.

autofocus
Automatically focuses input.

onfocus=alert(1)
Executes when focused.

x="
Repairs broken HTML.

Execution Flow:
Page loads → autofocus triggers focus → onfocus executes.

8. JavaScript String Breakout

Single Quote Context:

```
var x = 'INPUT';
```

Payload:
' ;alert(1); //

Breakdown:

'
Closes original string.

;
Ends statement.

alert(1)
Executes JavaScript.

//
Comments remaining code.

Result:

```
var x = '';  
alert(1); //
```

This is one of the most important XSS techniques.

9. Double Quote JavaScript Breakout

Context:
var x = "INPUT";

Payload:
" ;alert(1); //

Same concept as single quote breakout.

Used when JavaScript string uses double quotes.

10. AngularJS Expression Injection

Detection Payload:
`{{1+1}}`

If page shows:
2

Then AngularJS is evaluating expressions.

Exploit Payload:
`{{constructor.constructor('alert(1'))()}}`

How It Works:

`constructor.constructor`
Accesses JavaScript Function constructor.

Equivalent To:
`Function('alert(1'))()`

Final Result:
`alert(1)` executes.

Important:
Works mainly in old AngularJS (1.x).

11. JSON + eval() Injection

Dangerous Pattern:
`eval(response)`

Payload:
`\"-alert(1)}//`

Why It Works:
Quotes are escaped but backslashes are not.

Double backslashes cancel escaping.

Result:
String closes early and JavaScript executes.

This is escape-sequence manipulation.

12. Special HTML Context Breakouts

Textarea Context:
`</textarea>`

Title Context:
`</title>`

Select Context:
`</select>`

Option Context:
`</option>`

Why Needed?
Browser ignores many tags inside special elements.

Must escape parent tag first before injecting executable HTML.

13. Parser Manipulation Tricks

Payload:
<>

Purpose:
Confuse browser parser.

How It Works:
Malformed HTML forces parser recovery.

Parser exits previous state and parses img tag correctly.

Useful In:

- Broken HTML
- Weird parsing states
- Filter bypass situations

14. Dangerous DOM Sinks

Very Dangerous:

- innerHTML
- outerHTML
- document.write()
- eval()
- insertAdjacentHTML()

Why Dangerous?
These parse strings as HTML or JavaScript.

Example:
element.innerHTML = userInput;

If attacker controls userInput:
HTML executes.

15. Common Event Handlers

onerror
Triggers when loading fails.

onload
Triggers when element loads.

onfocus
Triggers on focus.

onmouseover
Triggers on hover.

onclick
Triggers on click.

Event handlers are extremely important in XSS.

16. Real Bug Hunter Workflow

Step 1:
Inject test string.

Step 2:
Check reflection.

Step 3:
Identify context.

Step 4:
Check escaping.

Step 5:

Find dangerous sink.

Step 6:
Choose payload according to context.

Step 7:
Test parser behavior.

This workflow is how professional hunters approach XSS.

17. Browser Parsing Concepts

Browsers automatically repair malformed HTML.

This behavior creates:

- parser breakouts
- context confusion
- filter bypasses

Special parser contexts:

- textarea
- title
- select
- script
- style

Understanding browser parsing is critical for advanced XSS.

18. Important Golden Rules

Rule 1:
Payload depends on context.

Rule 2:
XSS = escape current context + execute JavaScript.

Rule 3:
Never memorize blindly.

Rule 4:
Understand parsing and browser behavior.

Rule 5:
Always inspect reflection carefully.